

Step 1 – Prompt Sensitivity and Baseline Optimization

The first step of the project focused on prompt sensitivity. Prompt sensitivity refers to the phenomenon where small changes in prompt wording can lead to significant differences in model outputs. Because of this, the first goal of the project was to design a robust system prompt that could reliably generate high-quality diagnostic quizzes grounded strictly in lecture material.

To accomplish this, several prompt variations were tested and refined until a consistent and structured system prompt was produced. The final prompt enforces strict rules for output formatting, prohibits the use of outside knowledge, and ensures that all questions are grounded in the lecture notes provided as input. This prompt effectively acts as the baseline instruction set for the quiz generation system.

The hardened system prompt used in the system is shown below:

```
HARDENED_SYSTEM_PROMPT = """
```

```
You are a graduate teaching assistant generating a diagnostic quiz.
```

```
Constraints:
```

- Questions must be grounded strictly in the lecture notes
- Do not add external facts
- Maintain consistent academic formatting

```
Generate:
```

- Three MCQ labeled Easy, Medium, Hard
- Two Short Answer Questions
- One Application Question

```
Include answers and explanations.
```

```
"""
```

Once the prompt was finalized, a baseline quiz generation function was implemented. This function sends the lecture notes along with the system prompt to the model and generates a structured quiz. The function below illustrates how quiz generation is performed using the baseline prompt.

```
def generate_quiz_baseline(notes, temperature=0.3):
```

```
    user_prompt = f"""
```

Create a diagnostic quiz based ONLY on the lecture notes below.

Requirements:

- 3 Multiple Choice Questions (Easy, Medium, Hard)
- 2 Short Answer Questions
- 1 Application Question
- Include answers and explanations
- Do NOT use outside knowledge

Lecture Notes:

{notes}

"""

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=temperature,
    messages=[
        {"role": "system", "content": HARDENED_SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt}
    ]
)

return response.choices[0].message.content
```

This baseline prompt and generation method served as the foundation for all subsequent experimentation and evaluation.

Step 2 – Dataset Construction

The second step of the project involved building a structured dataset that would allow us to test the robustness of our prompt and generation pipeline. The dataset was designed to contain three different categories of examples: typical cases, edge cases, and adversarial cases.

Typical cases represent normal lecture content extracted from course materials. These are the primary inputs the system is expected to handle in real use. Edge cases represent unusual or difficult inputs such as incomplete explanations, very short lecture segments, or ambiguous content that may challenge the model's reasoning ability. Adversarial cases are deliberately crafted to test the model's resistance to hallucination or instruction manipulation. These inputs attempt to make the model ignore grounding instructions or introduce external information.

Each dataset item was structured with consistent metadata so it could easily be processed by later components in the pipeline. Each entry includes the lecture name, the chunk identifier, the lecture notes text, and the case type classification.

An example dataset entry looks like this:

```
{
  "lecture": "Module 5 Prompting Techniques",
  "chunk_id": "Module5_chunk_12",
  "notes": "lecture text here",
  "case_type": "typical"
}
```

The dataset was then assembled by combining all case types into a single structured dataset.

```
DOCUMENTED_DATASET = typical_docs + edge_cases + adversarial_cases
print("Full dataset size:", len(DOCUMENTED_DATASET))
```

The resulting dataset contained over two hundred lecture chunks and served as the primary evaluation and experimentation dataset for the rest of the project.

Step 3 – Fine-Tuning Dataset Creation and RAG Pipeline

The third step introduced retrieval augmented generation (RAG) and preparation of a fine-tuning dataset. The purpose of RAG is to reduce hallucinations and ensure that the model's answers are grounded in relevant lecture content. Instead of relying purely on the model's internal knowledge, the system retrieves relevant lecture chunks from a vector database and provides them as context during generation.

For the vector database, the project uses Qdrant. Lecture notes were converted into embeddings and stored in Qdrant along with metadata identifying the lecture and chunk source.

The following code snippet illustrates how lecture chunks were embedded and stored in the vector database.

```
sample_docs = typical_docs[:40]
```

```
points = []
```

```
for item in sample_docs:
```

```

embedding = get_embedding(item["notes"])

points.append(
    PointStruct(
        id=str(uuid.uuid4()),
        vector=embedding,
        payload={
            "lecture": item["lecture"],
            "chunk_id": item["chunk_id"],
            "notes": item["notes"]
        }
    )
)

qdrant.upsert(
    collection_name=collection_name,
    points=points
)

```

Once stored, the system retrieves relevant lecture chunks using semantic similarity search. When a query is submitted, the query is embedded and compared against stored vectors to retrieve the most relevant lecture content.

```

def retrieve_relevant_chunks(query, top_k=3):

    query_embedding = get_embedding(query)

    results = qdrant.search(
        collection_name=collection_name,
        query_vector=query_embedding,
        limit=top_k
    )

    return results

```

These retrieved chunks are then inserted into the generation prompt to provide grounded context for quiz generation. This retrieval step ensures that all questions are based directly on lecture material rather than model assumptions.

The RAG-based quiz generation function looks like this:

```

def generate_quiz_with_rag(query):

    chunks = retrieve_relevant_chunks(query)
    context = "\n\n".join([c.payload["notes"] for c in chunks])

```

```
prompt = f"""
Use the lecture notes below to create a diagnostic quiz.
```

Lecture Notes:

```
{context}
"""
```

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=0.3,
    messages=[
        {"role": "system", "content": HARDENED_SYSTEM_PROMPT},
        {"role": "user", "content": prompt}
    ]
)

return response.choices[0].message.content
```

In addition to RAG, Step 3 also created a small fine-tuning dataset. Training examples were generated automatically using the baseline quiz generator so that they could later be used for fine-tuning a model if needed.

```
fine_tune_examples = []

for item in fine_tune_source[:20]:
    quiz_output = generate_quiz_baseline(item["notes"], temperature=0.2)

    fine_tune_examples.append({
        "system": HARDENED_SYSTEM_PROMPT,
        "user": f"""
```

Create a diagnostic quiz based ONLY on the lecture notes below.

Lecture Notes:

```
{item["notes"]}
""",
    "assistant": quiz_output
})
```

These examples follow the chat training format required for modern fine-tuning workflows.

Step 4 – Meta Prompting and Evaluation

The final step of the pipeline focused on improving the prompt using meta prompting and evaluating the resulting prompt using both qualitative task metrics and quantitative perplexity.

Meta prompting allows the model to critique its own instructions. The system prompt was passed to the model with instructions to identify strengths, weaknesses, and potential improvements. This critique was then used to generate an improved version of the system prompt.

The critique function used for meta prompting is shown below:

```
def critique_system_prompt(system_prompt):
```

```
    critique_prompt = f"""
    Review the following system prompt.
```

```
    Identify:
```

- strengths
- weaknesses
- improvements

```
    Prompt:
```

```
    {system_prompt}
    """
```

```
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0.2,
        messages=[
            {"role": "system", "content": "You are a prompt engineering expert."},
            {"role": "user", "content": critique_prompt}
        ]
    )
```

```
    return response.choices[0].message.content
```

Using the critique generated above, a revised prompt was produced through structured prompt improvement.

```
def improve_system_prompt(system_prompt, critique):
```

```
    prompt = f"""
    Improve the system prompt based on the critique.
```

Prompt:
{system_prompt}

Critique:
{critique}
"""

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=0.2,
    messages=[
        {"role": "system", "content": "You are a prompt engineering expert."},
        {"role": "user", "content": prompt}
    ]
)

return response.choices[0].message.content
```

To evaluate prompt performance, several task metrics were used including formatting correctness, grounding in lecture notes, clarity of questions, and hallucination risk. These metrics were assessed using a secondary evaluation prompt.

Finally, quantitative evaluation was performed using perplexity. Perplexity measures how predictable a model's output is, with lower perplexity indicating more confident and stable predictions.

Perplexity was calculated using token log probabilities returned by the model.

def generate_with_logprobs(system_prompt, notes):

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=0,
    logprobs=True,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": notes}
    ]
)

token_items = response.choices[0].logprobs.content
logprobs = [t.logprob for t in token_items]

avg_neg_logprob = -sum(logprobs) / len(logprobs)
```

```
perplexity = math.exp(avg_neg_logprob)
```

```
return perplexity
```

The final evaluation produced the following results:

Baseline prompt average perplexity: **1.2558**

Revised prompt average perplexity: **1.2736**

These results indicate that while the revised prompt improved structural clarity, the baseline prompt achieved slightly lower perplexity, meaning it produced marginally more predictable outputs.

Final Pipeline Overview

At the end of these four steps, the system includes a complete prompt engineering workflow. The system begins with a hardened system prompt optimized through prompt sensitivity testing. A structured dataset containing typical, edge, and adversarial inputs is used to test the model's robustness. Lecture material is stored in a Qdrant vector database and retrieved during generation using a RAG pipeline to ensure outputs remain grounded in course material. A small fine-tuning dataset has also been prepared for future experimentation. Finally, the prompt was refined using meta prompting and evaluated using both qualitative task metrics and quantitative perplexity.

This pipeline ensures that the quiz generation system is grounded, robust, and empirically evaluated.